

Designing an LLM Inference Serving Platform

Serving a fleet of models across sizes and versions with canary rollouts, A/B testing, and instant rollback — a system design interview walkthrough with a deep dive on scaling.

Codename “Conduit” · control-plane + data-plane architecture

1. Problem statement

We operate a **fleet** of large language models — several families (e.g. a 120B flagship, a 70B workhorse, a 24B mid-tier, an 8B fast model, an embedding model), each shipping in multiple sizes and a continuous stream of fine-tuned/quantized *versions*. We need a serving platform that lets teams push new versions to production traffic **safely**: gradually, with measurement, and with the ability to undo instantly.

The interview framing: design the system that sits between clients and GPU model servers and owns *how much traffic each version receives*, plus the machinery to change that safely at scale.

Functional requirements

- Serve inference (chat/completion, streaming, embeddings) for many models and versions behind stable, logical endpoints.
- **Canary rollouts**: shift traffic to a new version in stages (1% → 5% → 25% → 50% → 100%) gated on live health metrics; auto-promote or auto-hold.
- **A/B testing**: run two (or more) versions on a controlled traffic split and compare quality, latency, and cost with statistical rigor.
- **Instant rollback**: revert to a known-good version in seconds without a redeploy.

- **Traffic routing:** rule-based routing by tenant/tier, region, prompt characteristics; weighted splits.

Non-functional requirements

Property	Target
Availability	99.95%+ on the routing path; a version failure must not take down the endpoint.
Routing overhead	< 5 ms p99 added latency vs. hitting a model server directly.
Rollback time	< 2 s from decision to 100% traffic on the previous version.
Config propagation	Routing changes reach every router replica in < 1 s.
Scale	Tens of thousands of RPS, thousands of GPUs, multi-region.

2. Back-of-the-envelope estimation

Assume peak **20,000 inference RPS** across the fleet, average 500 input tokens and 300 output tokens per request.

- **Token throughput:** $20k \times 300 = 6M$ output tokens/s generated fleet-wide (plus ~10M input tokens/s prefilled).
- **GPU sizing:** a 70B model on an 8×H100 node sustains ~5–8k output tok/s with continuous batching. Serving 6M tok/s across mixed sizes $\Rightarrow$ on the order of **1,000–2,000 GPUs**, weighted toward the large models.
- **Router fleet:** a router replica handles ~2–5k RPS of thin proxying $\Rightarrow$ ~10–20 replicas per region for headroom, stateless and cheap.

- **Telemetry:** ~20k events/s × (metrics + sampled traces) ⇒ a streaming aggregation pipeline, not per-request writes to a DB.
- **Cost signal:** at ~\$2/1M tokens for the flagship, fleet spend is dominated by the large models — cost must be a first-class rollout gate.

Key insight: the data plane (GPUs) is enormous and expensive; the control plane (routing decisions, config, gates) is tiny by comparison. Keep the control plane *off the hot path* — routers cache config locally and never make a synchronous call to the control plane to serve a request.

3. High-level architecture

Two planes. The **data plane** serves tokens; the **control plane** decides policy and observes health.

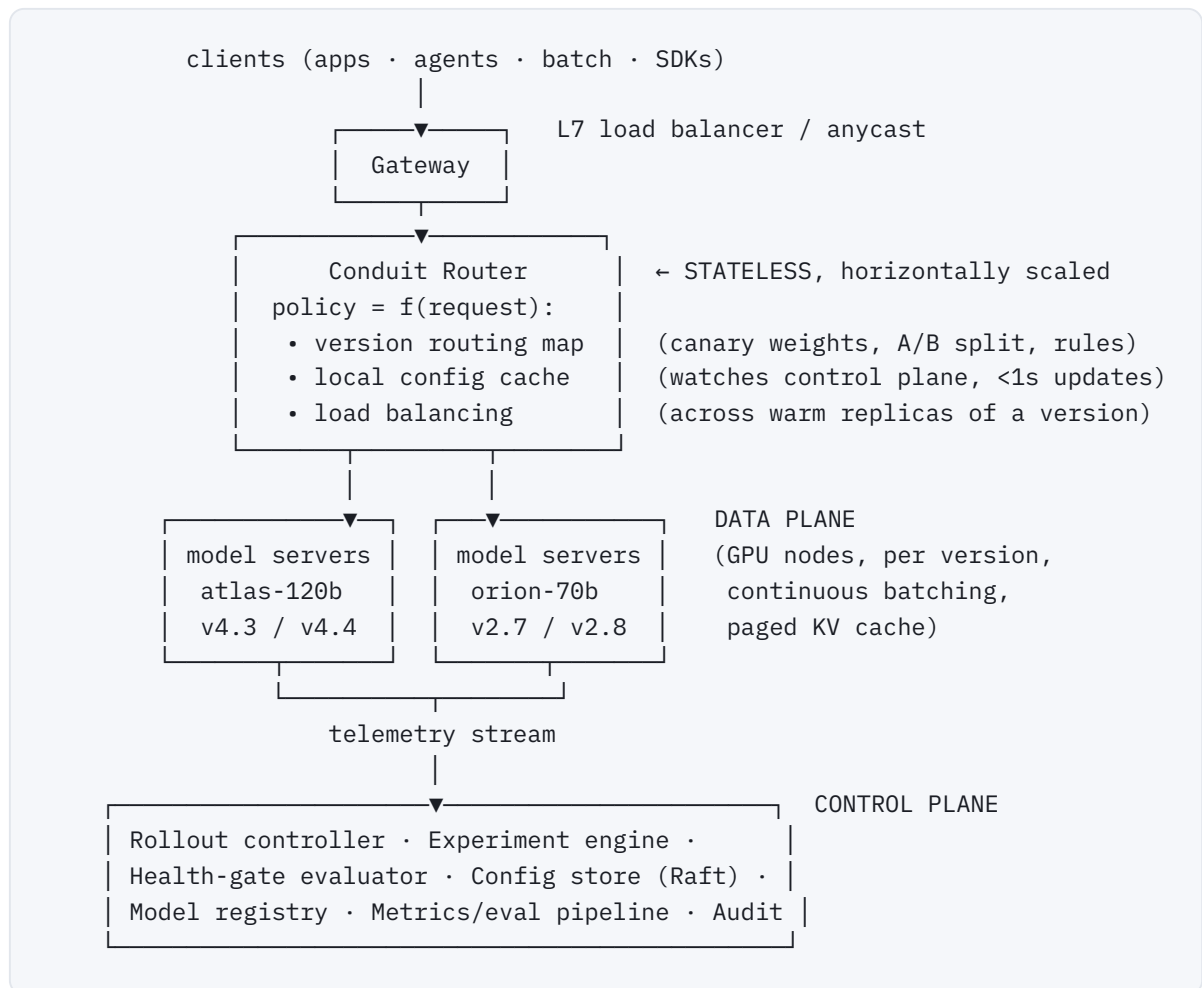


Figure 1 – Request path (top-down) and control/telemetry loop.

Component responsibilities

Component	Responsibility
Router	Resolve logical endpoint → concrete version → warm replica; apply weights/rules; stream tokens back. Emits telemetry.
Model registry	Immutable catalog of versions (weights digest, engine config, hardware profile). Source of truth for “what can be served.”
Config store	Strongly-consistent (Raft/etcd) store of the active routing map. Routers watch it; changes fan out in <1s.
Rollout controller	State machine that advances canary stages when gates pass and holds/reverts when they fail.
Experiment engine	Assigns requests to A/B arms, tracks metrics, computes significance.
Health-gate evaluator	Windowed evaluation of latency/error/quality/cost/feedback signals against thresholds.
Telemetry pipeline	Streaming aggregation of per-request metrics into rollout/experiment windows.

4. API design

Clients call a *logical* model name; the platform hides which physical version serves them. This indirection is what makes rollouts and rollback invisible to callers.

```

# Data plane – inference (client-facing, version-agnostic)
POST /v1/infer
  { "model": "orion-70b", "messages": [...], "stream": true }
  → 200 headers: x-served-version: v2.8, x-request-id, x-experiment-arm: B

# Control plane – rollouts
POST /control/rollouts      { model, target_version, stages, gates }
POST /control/rollouts/{id}:advance | :hold | :rollback
GET  /control/rollouts/{id} → stage, gate status, history

# Control plane – experiments & routing
POST /control/experiments  { model, arms:[{version,weight}], metrics }
GET  /control/experiments/{id} → per-arm metrics, deltas, significance
PUT  /control/routing      { rules:[...], default_version }

```

Figure 2 – The `x-served-version` response header is the audit hook that ties every response to a concrete version.

5. Canary rollouts

A rollout is a **state machine** owned by the rollout controller. Each stage carries a target traffic weight and a bake window; promotion is guarded by health gates evaluated over that window.

```

CREATED → 1% → 5% → 25% → 50% → 100% → COMPLETED
  every transition: wait bake window → evaluate gates
    gates pass → write new weight to config store → routers pick up
    gates fail → HOLD (page on-call) or ROLLBACK (auto)

```

Traffic weighting happens in the router: for each request it draws from a weighted distribution over eligible versions. Assignment is **hashed on a stable key** (user/session/tenant) so a given user sees a consistent version within a stage — avoiding flip-flopping mid-conversation.

Health gates (all must pass to promote)

- **Latency:** p95/p99, TTFT, tokens/sec within threshold vs. baseline.
- **Reliability:** error rate and timeout rate below ceiling.
- **Quality:** eval win-rate / groundedness on live-sampled traffic $\geq$ baseline.
- **Cost:** cost per 1M tokens within budget.
- **Feedback:** thumbs-up rate, regeneration/abandonment rate acceptable.

6. A/B testing

A/B differs from canary in intent: canary asks “is the new version *safe?*”, A/B asks “is it *better?*”. The engine holds a fixed split (e.g. 50/50) for long enough to reach significance rather than ramping.

- **Deterministic assignment:** `arm = hash(experiment_id, unit_id) % 100` keeps a unit sticky to its arm and makes assignment reproducible.
- **Metrics:** per-arm win-rate, groundedness, latency, TTFT, cost, feedback — with confidence intervals and a significance test (e.g. two-proportion z-test / sequential testing to allow early stopping).
- **Guardrails:** even a “test” arm is gated — if the challenger breaches a safety gate, it is pulled regardless of the experiment.
- **Handoff:** a winning arm graduates directly into a canary rollout to 100%.

7. Instant rollback

Rollback is fast **because it is just a config write**, not a deploy. Three properties make sub-2s reversion possible:

1. **Versions are immutable and kept warm.** The previous version's replicas are never torn down during a rollout, so reverting needs no cold start.
2. **Routing is data, not code.** Reverting writes the old weights to the config store; routers watching it swap within their propagation SLA (<1s).
3. **In-flight requests drain.** New requests route to the good version immediately; streaming requests already on the bad version finish or are retried per policy.

Automatic vs. manual: the gate evaluator can trigger rollback on a hard breach (error spike) without a human — the same mechanism a person clicks in the console. The button and the automation share one code path.

8. Data model & storage

Data	Store	Why
Active routing map	etcd / Raft KV	Strong consistency + watch/notify for <1s fan-out.
Model registry & rollout history	Relational (Postgres)	Transactional metadata, joins for the console, audit.
Model weights	Object store + node-local cache	Large immutable blobs, content-addressed by digest.
Metrics / gates	Time-series DB	Windowed aggregates for gate evaluation and dashboards.
Request logs / traces	Columnar lake (sampled)	Offline eval, experiment analysis, debugging.

9. Deep dive: scaling the system

This is the heart of the design. Inference is unusually expensive and unusually bursty, so scaling touches every layer: the stateless router tier, the GPU data plane, the control/telemetry pipeline, and the multi-region topology.

9.1 Scaling the router tier (stateless, easy)

- Routers hold no per-request state, so they scale **horizontally** behind an L7 LB; add replicas on RPS/CPU.
- Each router keeps a **local, watched copy** of the routing map — routing decisions are pure functions of the request plus in-memory config, so there is no control-plane call on the hot path.
- Use **consistent hashing** from router → model-server replicas so that requests sharing a prompt prefix (or a session) land on the same replica and reuse its **KV/prefix cache**.
- The gateway terminates TLS, does auth/rate-limiting, and buffers streaming responses so slow clients don't pin GPU workers.

9.2 Scaling the GPU data plane (the hard part)

GPUs dominate cost and are the real bottleneck. Throughput per GPU — not replica count — is the lever.

- **Continuous (in-flight) batching:** the engine admits new requests into a running batch every decode step instead of waiting for a batch to fill. This is the single biggest throughput win; it trades a little TTFT for large gains in tokens/sec.
- **Paged KV cache:** attention keys/values are stored in fixed-size pages (PagedAttention-style) so memory isn't fragmented by variable sequence lengths — more concurrent sequences per GPU.

- **Prefix caching:** shared system prompts / few-shot preambles are prefilled once and reused across requests — huge for agent workloads with long, repeated prompts.
- **Prefill/decode disaggregation:** prefill is compute-bound and bursty; decode is memory-bandwidth-bound and steady. Splitting them onto separate pools lets each scale independently and keeps a big prefill from stalling decodes.
- **Model parallelism for large models:** the 120B flagship is sharded with **tensor parallelism** within a node and **pipeline parallelism** across nodes; small models run one-per-GPU. Hardware profile is part of the registry entry.
- **Quantization & right-sizing:** serving FP8/INT8 versions where quality gates allow cuts GPU count materially; the routing layer can even send cheap traffic to a smaller model via rules.

9.3 Autoscaling model servers

Scale on inference-native signals, **not CPU**. The right signals are **queue depth / pending tokens** and **TTFT vs. SLO**. Cold starts are the enemy — loading tens of GB of weights takes tens of seconds — so:

- Keep a **warm pool** of standby replicas per hot version; scale from warm, not cold.
- Cache weights on **node-local NVMe** (content-addressed by digest) so a new replica loads from local disk, not object storage.
- Use **predictive / scheduled** scaling for known diurnal peaks; reactive scaling only for surprises.
- Scale down conservatively — GPUs are costly, but flapping under bursty traffic wastes cold-start capacity.

9.4 Handling surges & overload

- **Admission control & priority queues:** shed or delay low-priority (batch) traffic first; protect interactive traffic's TTFT.
- **Load shedding with backpressure:** return 429 with retry hints before queues grow unbounded; a full queue that never drains is worse than a fast rejection.

- **Graceful degradation via routing:** under pressure, rules can down-route spillover from a saturated large model to a smaller one rather than failing.
- **Caching:** an exact-match cache (and optionally a semantic cache with an embedding lookup) absorbs repeated queries before they reach a GPU.

9.5 Scaling the control & telemetry plane

- **Off the hot path:** config is read from a local cache; the control plane can be briefly unavailable without dropping traffic (routers keep serving last-known config).
- **Telemetry as a stream:** routers emit metrics to a bus (Kafka-style); a streaming job pre-aggregates into per-version/per-window counters. Gate evaluation reads aggregates, never raw per-request rows — this is what lets gates scale to 20k RPS.
- **Sampling:** full metrics, sampled traces, and sampled request/response bodies for eval — you cannot store every token stream at fleet scale.
- **Config store limits:** Raft stores are not built for high write throughput, but routing changes are rare (a few per rollout) so this is fine — keep only the routing map there, not metrics.

9.6 Multi-region

- **Region-local serving:** route users to their nearest region for latency; GPU pools and routers are per-region.
- **Data residency:** routing rules pin regulated traffic (e.g. EU) to in-region models — a first-class rule, not an afterthought.
- **Control-plane replication:** the config store replicates globally; a rollout can be per-region (canary in one region first) or global.
- **Failover:** if a region loses capacity, the gateway spills traffic to the next region, subject to residency constraints and added latency.

9.7 Cost optimization

At fleet scale, GPU hours are the dominant line item, and cost per 1M tokens is a first-class rollout gate — a version that regresses cost is treated like one that regresses latency.

Optimization runs across four levers: **use fewer GPUs, use cheaper GPUs, do less work, and buy capacity smarter.**

Raise utilization (fewer GPUs for the same load)

- **Continuous batching + high goodput:** the same throughput levers from §9.2 are cost levers — every point of GPU utilization won is a GPU not bought. Target a healthy utilization band (e.g. 60–75%) that balances cost against burst headroom.
- **Bin-packing & multiplexing:** co-locate small models (8B, embeddings) on shared GPUs; pack replicas to fill memory rather than stranding it. Reclaim fragmented KV memory with paged attention.
- **Scale-to-warm, not scale-to-zero for hot versions; scale-to-zero for cold ones:** rarely-used versions (old canaries, niche fine-tunes) idle down to zero and cold-start on demand; only actively-served and rollback-target versions stay warm.

Do less work per request

- **Model right-sizing via routing:** the cheapest optimization is not running the flagship when a 24B model suffices. Rules and a small router-classifier send easy prompts to smaller/cheaper models and reserve the 120B for hard ones (cost-aware routing / cascades).
- **Quantization:** FP8/INT8 versions cut memory and boost throughput 1.5–2×; promote them through the same quality gates so cost wins never silently degrade output.
- **Prefix & response caching:** shared system prompts prefilled once; exact/semantic caches serve repeated queries with zero GPU cost. For agent workloads with long static preambles this is often the single largest saving.
- **Speculative decoding:** a small draft model proposes tokens the large model verifies in parallel — more tokens per decode step, lower cost per token at equal quality.
- **Prompt/output discipline:** cap max output tokens per tier, trim redundant context, and price long contexts explicitly so callers internalize the cost.

Buy capacity smarter

- **Tiered capacity:** reserved/committed GPUs for the steady baseline, on-demand for the predictable diurnal peak, and spot/preemptible for interruptible batch work (which drains first under overload anyway).
- **Right hardware for the job:** memory-bandwidth-bound decode may run cost-effectively on cheaper accelerators than compute-bound prefill — another payoff of prefill/decode disaggregation.
- **Follow-the-sun packing:** shift interruptible/batch load to regions in their off-peak trough to reuse capacity already paid for.

Make cost observable & accountable

- **Per-version, per-tenant cost attribution:** the telemetry pipeline computes \$/1M tokens and \$/request by version and tenant, so rollouts, experiments, and chargeback all share one cost signal.
- **Cost as a gate & a guardrail:** a canary that raises cost beyond budget holds automatically; per-tenant budgets and rate limits prevent a single caller from blowing up spend.
- **Idle-warm-pool accounting:** the cost of standby replicas (the price of instant rollback and surge safety) is measured explicitly so the safety/cost tradeoff is a deliberate dial, not a surprise.

Rule of thumb: the optimizations compound multiplicatively — cost-aware routing (send 60% of traffic to a model 5× cheaper) usually dwarfs any single kernel-level win. Optimize the *routing mix* first, then per-GPU throughput, then procurement.

Bottlenecks & how we scale past them

Bottleneck	Scaling move
GPU memory (KV cache)	Paged attention, prefix cache reuse, quantization.
GPU compute (prefill)	Prefill/decode disaggregation, chunked prefill.
Cold-start latency	Warm pools, node-local weight cache, predictive scale.
Telemetry volume	Streaming pre-aggregation, sampling.
Config fan-out	Watch-based push from Raft store, local caches.
Cross-region latency	Region-local pools, anycast, residency-aware routing.

10. Failure modes & tradeoffs

Failure / decision	Handling / tradeoff
Control plane down	Routers serve last-known config; no new rollouts, but traffic is unaffected.
Bad version deployed	Gates catch it at low % and auto-hold/rollback; blast radius bounded by stage.
Model server crash	Router retries another replica; health checks eject the dead one.
Consistency vs. speed	Config is strongly consistent (correctness of routing); metrics are eventually consistent (a slightly stale gate is acceptable).
Sticky assignment vs. even split	Hash-based stickiness gives consistent UX but slightly lumpy splits; acceptable at scale.
Warm pools vs. cost	Idle warm GPUs cost money but are the price of instant rollback and surge safety.

11. Summary

Conduit separates a tiny, strongly-consistent **control plane** from a massive, expensive **data plane**, and keeps the control plane off the request hot path. Safe delivery — canary, A/B, rollback — reduces to **writing weighted routing config** that stateless routers apply locally, while a streaming telemetry pipeline feeds health gates that promote or revert automatically. Scaling is won at the GPU layer (continuous batching, paged KV cache, prefill/decode disaggregation, warm-pool autoscaling) and made safe by region-local topology and aggressive-but-bounded overload handling.

Interview soundbite: “Routing is data, not code; versions are immutable and warm; gates are automatic. That combination is what turns a risky model deploy into a reversible config change.”